

NOVAC Internship – Task 2 Daily Documentation

Task: File Upload APIs, MongoDB Integration, Semantic Chunking, and Embedding Storage

Overview

Today's work focused on extending the previously built Flask and FastAPI API prototypes into a more advanced AI-oriented document processing pipeline. The task involved implementing file upload functionality, integrating MongoDB for database storage, performing semantic chunking using Natural Language Processing (NLP) techniques, generating embeddings using Sentence Transformers, and displaying processed chunks dynamically on the frontend.

The project evolved from simple GET and POST APIs into a mini document ingestion system capable of processing TXT and PDF files, extracting text, splitting content intelligently, generating vector embeddings, and storing structured chunk data in MongoDB.

Resources and References Used

- MongoDB Official Documentation
- MongoDB Compass
- Sentence Transformers Documentation
- Hugging Face Models
- NLTK Documentation
- PyPDF Documentation
- Flask Official Documentation
- FastAPI Official Documentation

Concepts Learned

MongoDB Integration: MongoDB is a NoSQL database used for storing unstructured and semi-structured data in the form of documents. MongoDB Compass was used to visually inspect stored chunk data and embeddings.

File Upload Handling: Learned how to upload files from the frontend to backend APIs using HTML forms and JavaScript FormData objects.

Semantic Chunking: Instead of splitting text based on fixed character limits, semantic chunking groups related sentences together based on meaning similarity. This improves contextual understanding during retrieval tasks.

Sentence Tokenization: NLTK's `sent_tokenize()` was used to split large paragraphs into individual sentences before semantic comparison.

Embeddings: Embeddings are numerical vector representations of text generated using AI models. They capture semantic meaning and are commonly used in Retrieval-Augmented Generation (RAG) systems and vector databases.

Sentence Transformers: The `all-MiniLM-L6-v2` Sentence Transformer model from Hugging Face was used to generate embeddings for sentences and chunks.

Cosine Similarity: Cosine similarity measures how semantically similar two sentence embeddings are. This was used to decide whether consecutive sentences belong to the same chunk.

PDF Text Extraction: Implemented PDF text extraction using the `pypdf` library to support document ingestion from PDF files.

AI Document Ingestion Pipeline: Understood the workflow of modern AI ingestion systems:

Document Upload → Text Extraction → Chunking → Embedding Generation → Database Storage.

Libraries, Modules, and Components Learned

Flask: Used for building lightweight backend APIs.

FastAPI: Used for creating structured and high-performance APIs.

pymongo: Used for connecting Python applications with MongoDB.

sentence-transformers: Used for loading pretrained embedding models from Hugging Face.

sklearn.metrics.pairwise.cosine_similarity: Used for semantic similarity comparison between sentence embeddings.

nltk: Used for sentence tokenization.

pypdf: Used for extracting text from PDF documents.

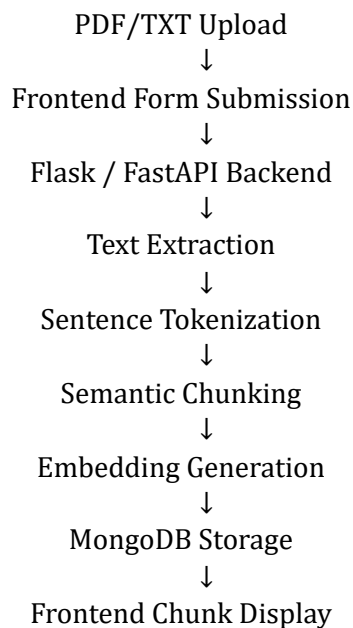
UploadFile and File: FastAPI utilities used for handling uploaded files.

FormData: JavaScript object used for sending files from frontend to backend.

Practical Implementation

- Installed MongoDB locally and connected it to Flask and FastAPI applications.
- Created file upload functionality in both Flask and FastAPI.
- Implemented TXT file reading and PDF text extraction.
- Used NLTK sentence tokenization for preprocessing text data.
- Generated semantic embeddings using Sentence Transformers.
- Implemented semantic chunking using cosine similarity thresholds.
- Stored chunk metadata, chunk IDs, filenames, and embeddings inside MongoDB.
- Displayed processed chunks dynamically on the frontend after upload.
- Used MongoDB Compass to verify stored documents and embeddings.
- Tested APIs on localhost using Flask and FastAPI servers.
- Debugged MongoDB connection issues, localhost access issues, and model dependency errors.

Architecture Understood



Challenges Faced

During implementation, several technical issues were encountered and resolved.

- MongoDB localhost binding and port conflicts occurred while starting mongod.
- Sentence Transformer imports initially failed because of TensorFlow and protobuf dependency conflicts.
- Browser localhost access issues caused temporary HTTP 403 errors.
- Semantic chunking logic initially produced incomplete chunks due to indentation and loop placement mistakes.
- PDF extraction support required handling multiple file types separately.
- Frontend rendering had to be updated after chunk metadata structure changed from plain text to JSON objects.

Resolving these issues helped improve understanding of debugging backend systems, dependency management, and API workflows.

Conclusion

Today's task significantly expanded the previous API prototypes into an AI-oriented backend processing system. By the end of the session, fully functional Flask and FastAPI applications were able to upload TXT and PDF files, extract text, perform semantic chunking, generate embeddings using Hugging Face models, store processed data inside MongoDB, and display stored chunks dynamically on the frontend.

The concepts learned today form the foundation of modern AI retrieval systems, vector databases, and Retrieval-Augmented Generation (RAG) pipelines.

Prepared By

Divesh Kumar S